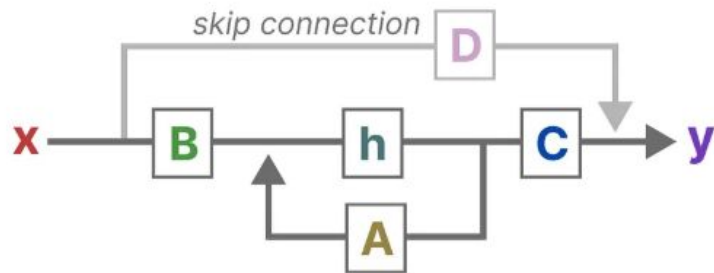
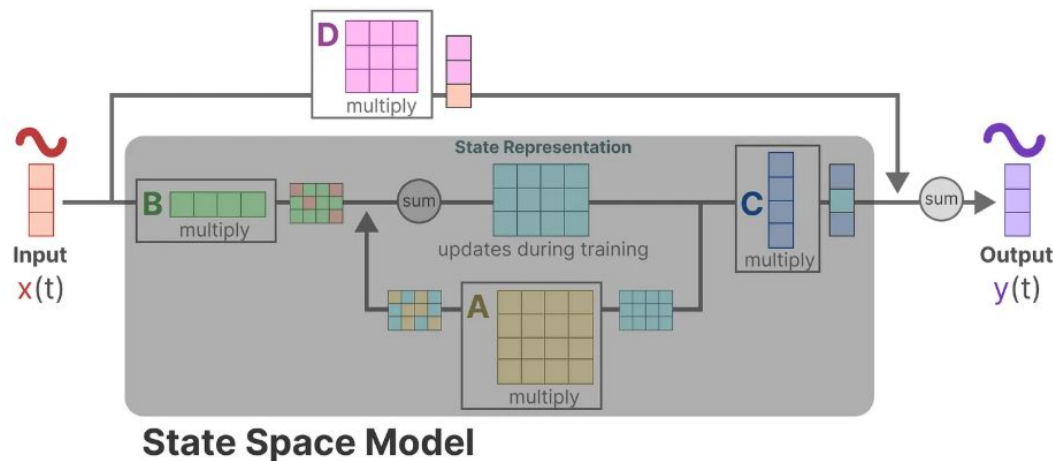


State Space Models

From continuous-time SSMs to selective, hardware-efficient sequence modeling.

What is a State Space Model?

An SSM models sequential data using a **hidden state**, **input**, and **output**.



Continuous-Time SSM

State equation:

$$h'(t) = Ah(t) + Bx(t)$$

Output equation:

$$y(t) = Ch(t) + Dx(t)$$

$x(t)$

Input sequence

$h(t)$

Hidden state

$y(t)$

Output

A

State transition matrix

B, C

Input / output projection

D

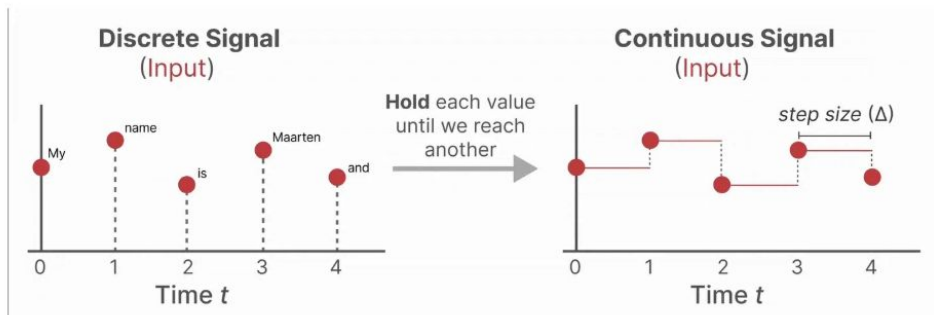
Skip connection

Discretization of SSMs

SSM equations are continuous — they must be converted to discrete form for computation.

Zero-Order Hold (ZOH)

Each input is held constant over a small interval, controlling sampling resolution.



Discretized Parameters

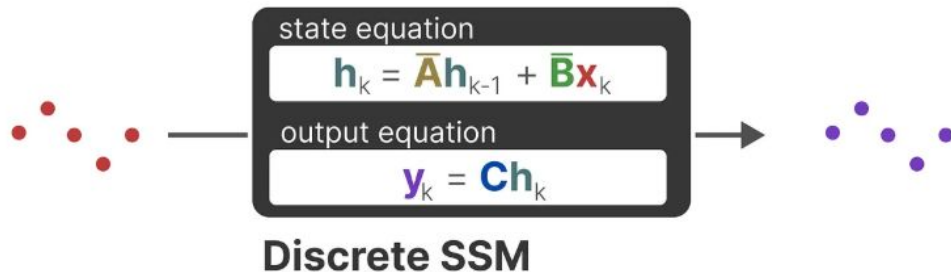
$$\bar{A} = e^{\Delta A}$$

$$\bar{B} = (\Delta A)^{-1} (e^{\Delta A} - I) \Delta B$$

Discrete SSM Equations

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k$$

$$y_k = Ch_k$$



Introduction to Mamba

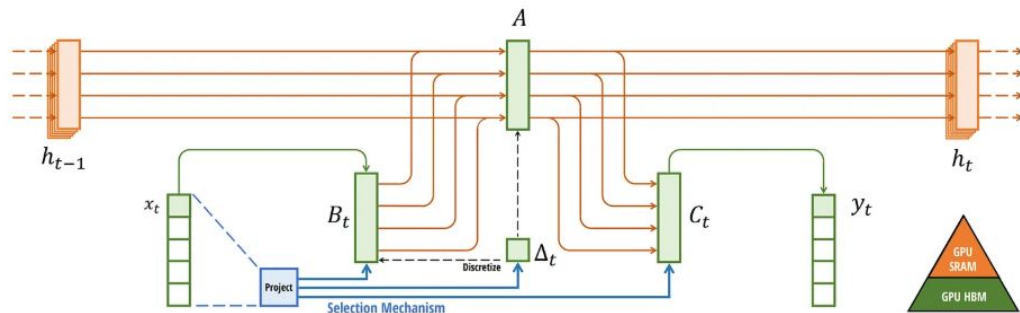
SELECTIVE SSM / S6

Mamba makes the SSM **selective** by replacing fixed parameters with input-dependent ones.



Selective Mechanism in Mamba

Dynamic Step Size $\Delta(x)$ Mamba uses input-dependent discretization to control memory updates.



Dynamic Parameters

$B = B(x)$, $C = C(x)$, $\Delta = \Delta(x)$
Matrix A stays fixed for stable state evolution.

Small Δ

Memory changes slowly —
retain previous context.

Large Δ

Memory updates strongly —
focus on current input.

Result: Efficient long-context reasoning.



What Mamba Learns

The model dynamically decides **what to store**,
what to ignore, and **when to update** memory.

Selective Scan Algorithm

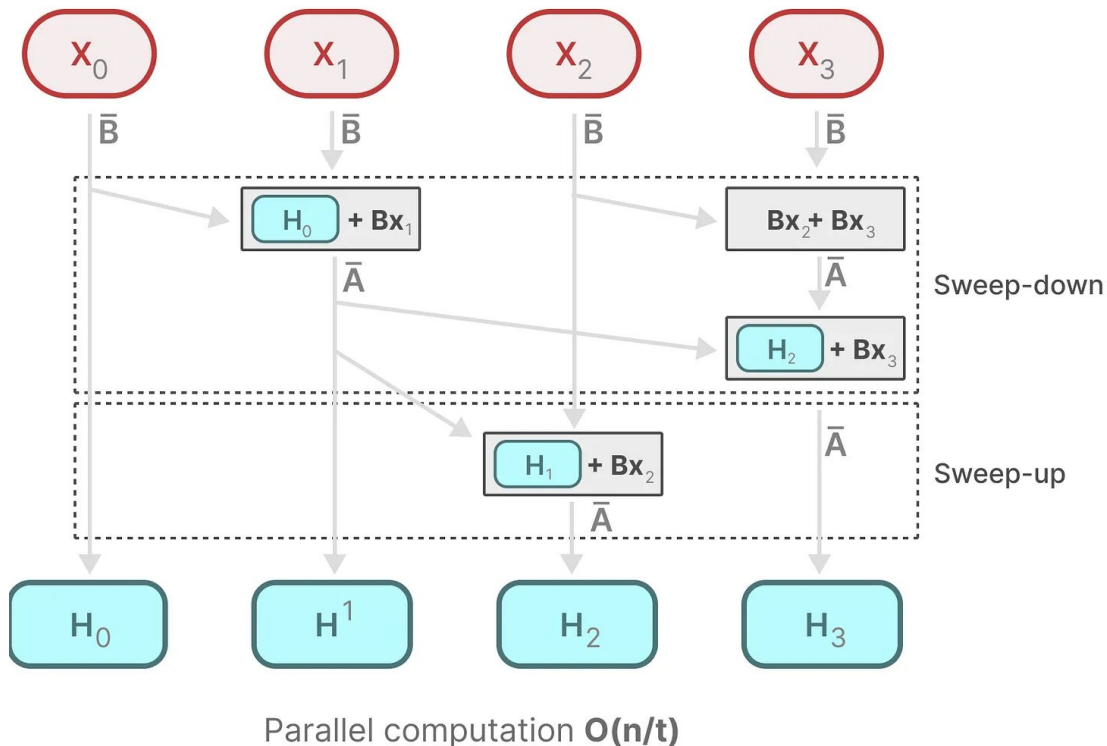
The Problem

Dynamic parameters destroy the convolution structure — fixed kernels can no longer be used. The state update becomes inherently sequential:

$$h_k = \bar{A}_k h_{k-1} + \bar{B}_k x_k$$

Mamba's Solution

Parallel Selective Scan — uses **associative scan operations** to parallelize what appears sequential.



Hardware-Aware Mamba Architecture

Modern GPUs face a memory bottleneck: **SRAM** is fast but small; **DRAM** is large but slow. Frequent transfers are expensive.

1 Kernel Fusion

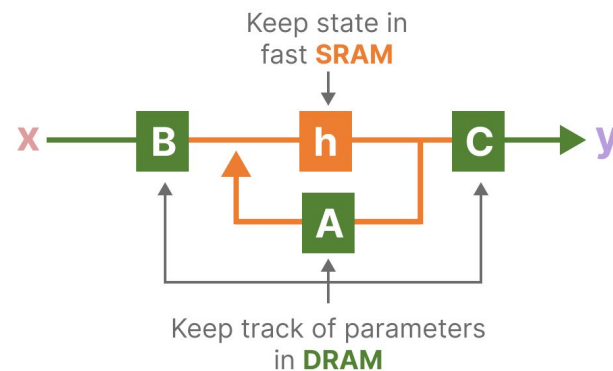
Combine discretization, selective scan, and output projection into **one GPU kernel**, avoiding intermediate tensor storage.

2 SRAM-Aware Computation

Keep hidden states in fast **SRAM**; keep parameters in **DRAM**.

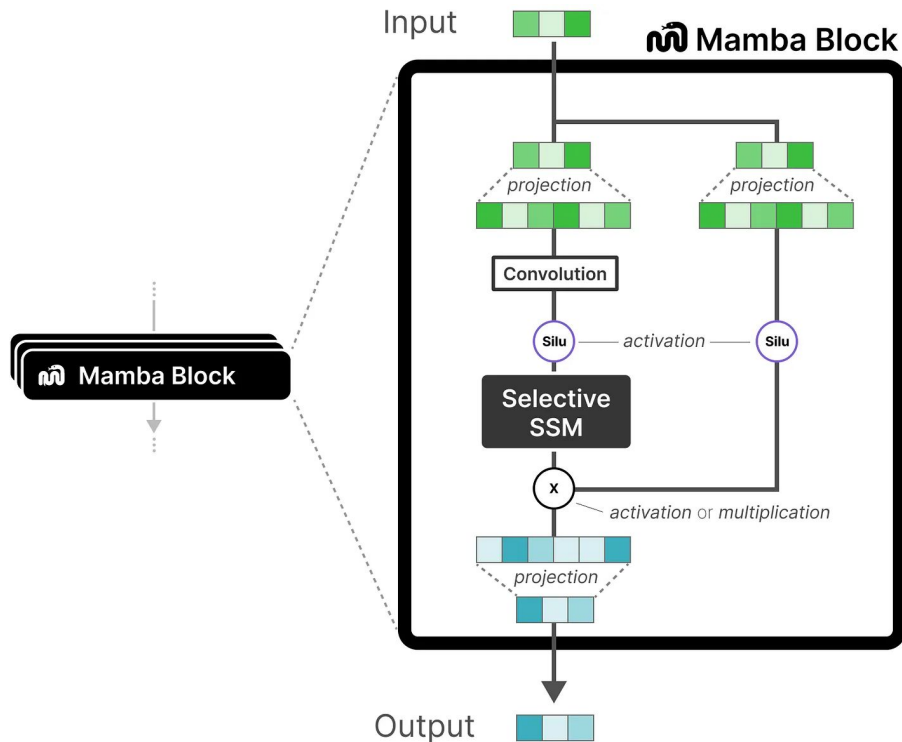
3 Recomputation

Intermediate states are recomputed during the backward pass instead of being read from **DRAM**.



Mamba Block Architecture

The Mamba block integrates selective SSM with standard Transformer components.



Key Design Points

- **Conv1D** before S6 captures local context
- **SiLU activation** introduces nonlinearity
- **Normalization** stabilizes training
- **Residual connection** preserves gradient flow

i This design balances local feature extraction with long-range selective memory.

Comparison & Summary

Attention

$O(N^2)$ — full context awareness

Standard SSM

$O(N)$ — fixed dynamics, poor selection

Mamba

$O(N)$ — selective, input-dependent

Key Takeaways

- SSMs model sequences via linear ODEs
- Discretization enables practical computation
- Mamba introduces input-dependent B, C, Δ
- Selective scan enables parallel training
- Hardware-aware design maximizes GPU efficiency

How VideoMamba Processes Video

A step-by-step walkthrough of VideoMamba's internal pipeline — from raw video input to final predictions. We trace how raw frames become semantic tokens, how spatial and temporal structure is encoded, and how the bidirectional Mamba blocks produce rich video representations for classification, retrieval, or detection.



Frame Sampling

From a long video, we **uniformly** (or **randomly**) select a fixed number of frames T to represent the whole video.

1 Input Video

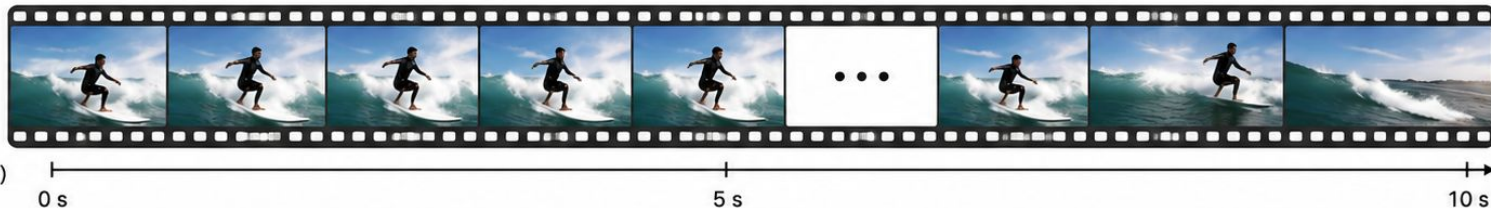


Long Video

Duration: 10 seconds

FPS: 30

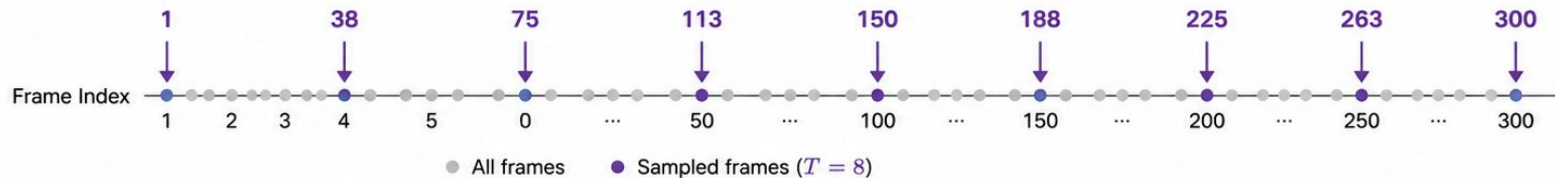
Total frames = 300



2 Sampling Strategy

Uniform Sampling

Select frames at regular intervals.



3 Sampled Frames ($T = 8$)

These 8 frames will be used for further processing.



Sampling reduces redundancy between adjacent frames, speeds up training/inference, and keeps important temporal information.

Frame Tensor & Why Sampling Matters

Frame Tensor Shape

Sampled frames are stacked into a 4D tensor:

 **(T, H, W, C) = (16, 224, 224, 3)**

Where **T** = frames, **H×W** = resolution, **C** = RGB channels.

Frame 101

Person slightly moves hand

Why Sparse Sampling?

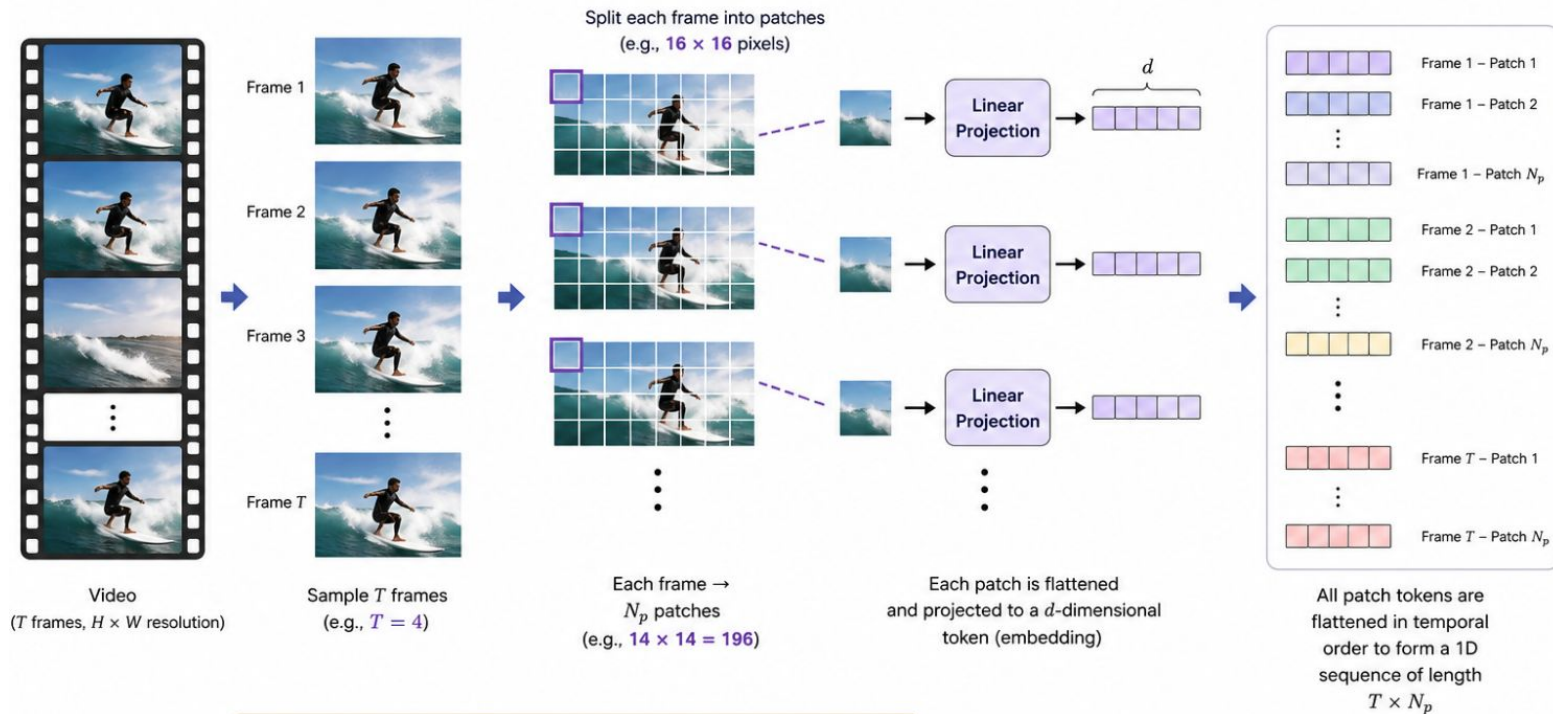
Videos are highly redundant — adjacent frames are often nearly identical.

Sparse sampling cuts computation while retaining motion signal.

Frame 102

Almost pixel-identical to 101

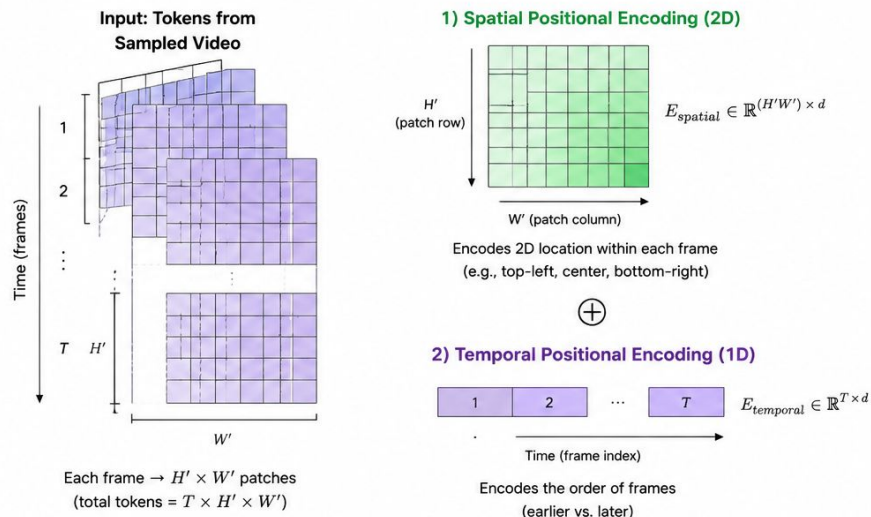
Patch Embedding



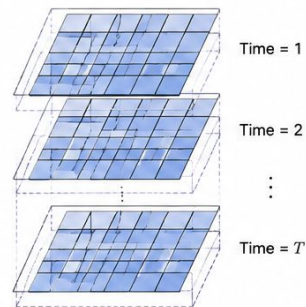
Temporal Positional Embeddings

Spatial and Temporal Encoding

Each token (patch) is augmented with spatial position (**where**) and temporal position (**when**) information.



3) Combined Spatio-Temporal Encoding



For each token (i, j, t) :

$$x'_{i,j,t} = x_{i,j,t} + E_{i,j}^{\text{spatial}} + E_t^{\text{temporal}}$$

where $x_{i,j,t} \in \mathbb{R}^d$ is the original patch token

Notation

- T : number of sampled frames
- H', W' : number of patches along height and width in a frame ($H' = H/P$, $W' = W/P$ for patch size P)
- d : embedding dimension

Intuition



Spatial encoding tells the model “where” a patch is in the frame.



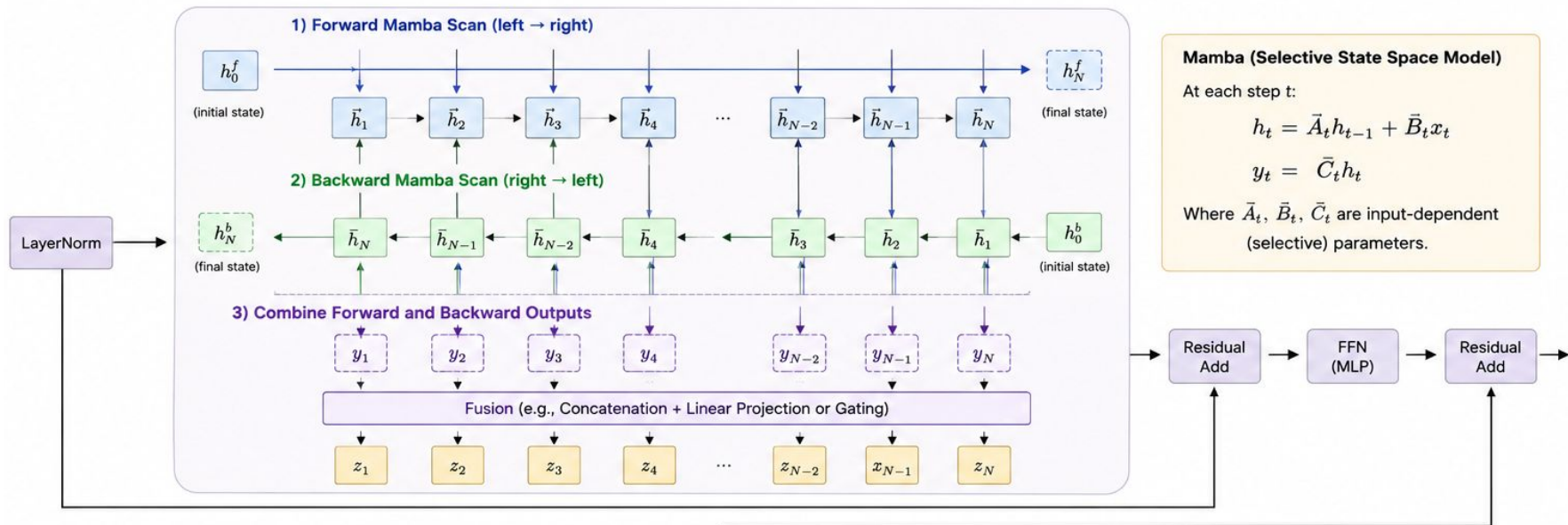
Temporal encoding tells the model “when” the frame/pattern occurs.

Result

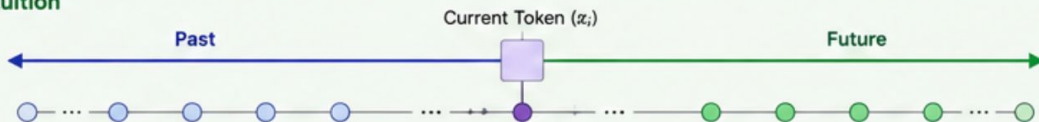


The model now knows both spatial location and temporal order for every token.

Bidirectional Mamba Blocks



Intuition



Bidirectional scan = full temporal context for understanding long-range dependencies.

Key Properties

- ✓ **Forward scan** summarizes information from the past.
- ✓ **Backward scan** summarizes information from the future.
- ✓ Every **output token z_i** has access to both left and right context.
- ✓ Still **linear complexity: $O(N)$** (each direction is a linear scan).

Feature Aggregation & Output Heads

Feature Aggregation

After Mamba blocks, token features are aggregated into a single video-level representation.

Task-Specific Heads

Classification

Linear → Softmax → action class

Retrieval

Project → normalized embedding

Detection

Temporal anchors → bounding boxes

Global Average Pooling

Average across all tokens

CLS Token

Use a dedicated [CLS] token

Experiment and Results

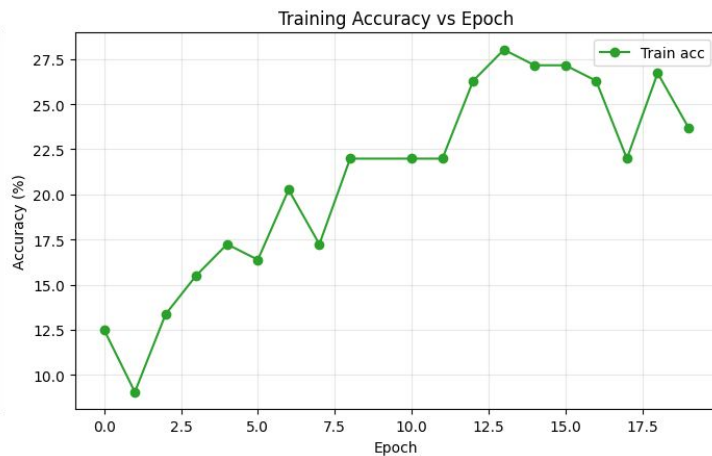
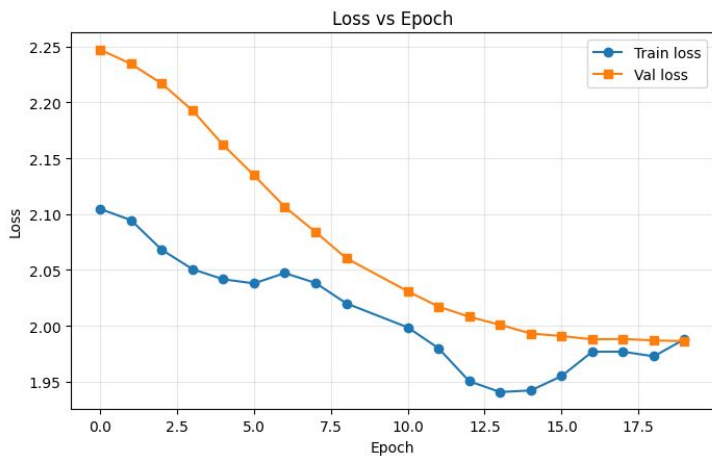


Model Configurations

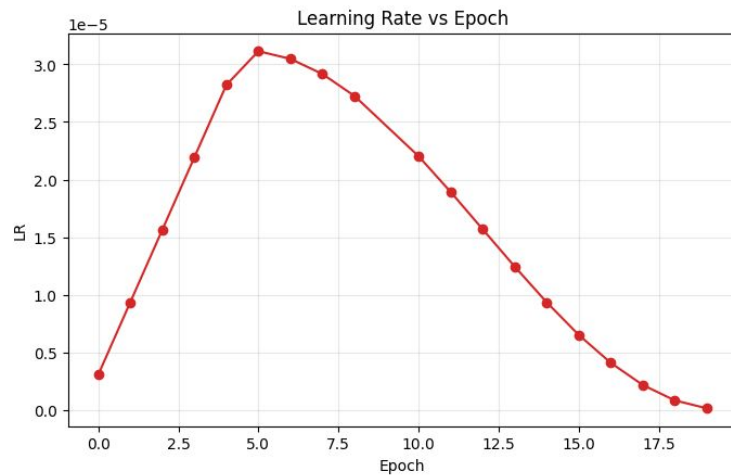
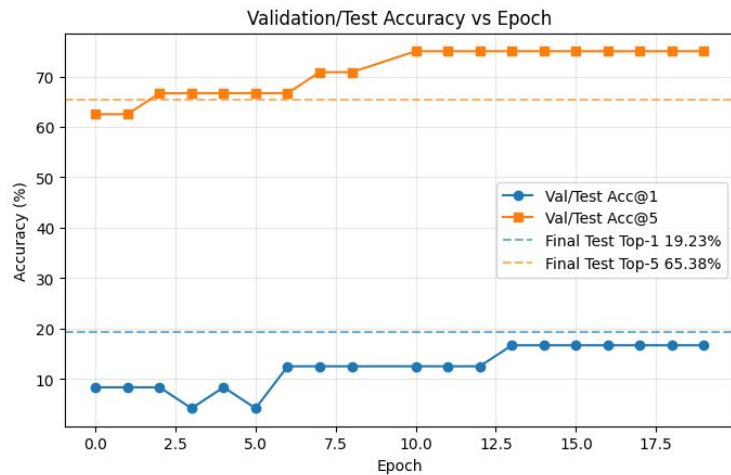
Model name -	vivim_tiny
Original pretraining -	Kinetics 400
Backbone status -	frozen
# Classes -	8
Embedding Dimension -	192
Custom Head -	Linear(192 -> 8)
Trainable params -	1544
Input Frames -	16
Input Resolution -	224x224
Epochs -	20



Results



Results



The Catch

